Path Replanning

Path Replanning

1. Course Content
2. Preparation
3. Path Replanning Case Demonstration
4. Source Code Analysis
5. Parameter Debugging

1. Course Content

Basic

- Start the road network navigation feature, and master using the roadnet_nav visualization panel to send a goal waypoint ID. Place an obstacle on the navigation path to block the current edge, and the robot will roll back to the last safe waypoint it passed and replan a traversable path to reach the goal waypoint.

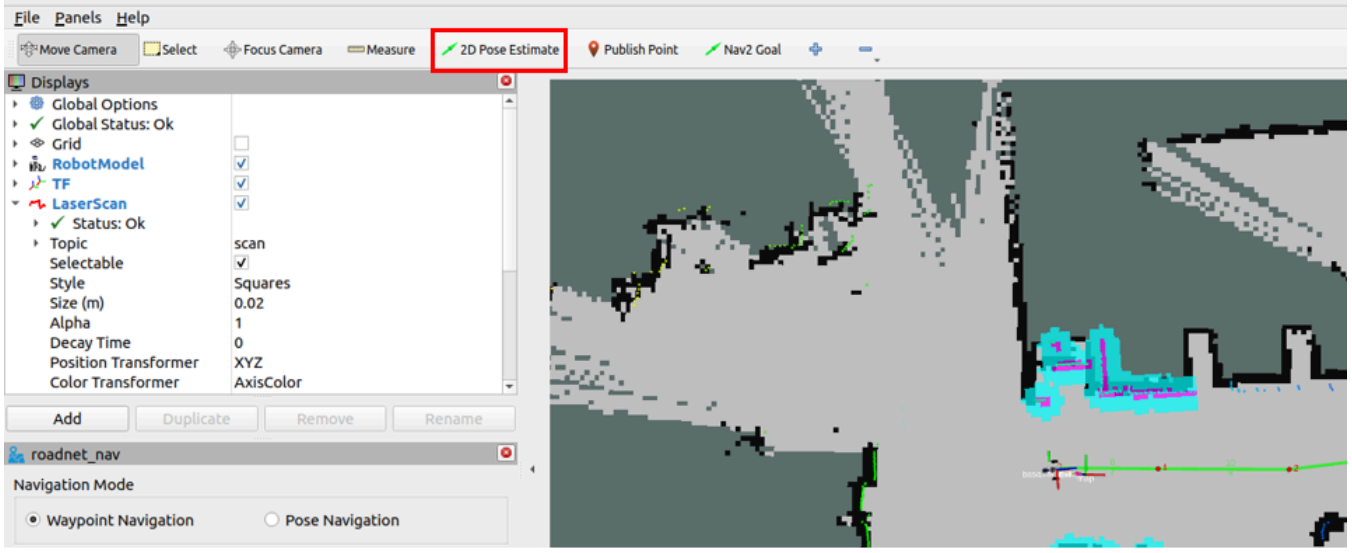
Advanced

- Understand the implementation logic of the program source code
- Master the parameter file configuration, being able to turn the path replanning feature off and on and to adjust the collision detection distance

2. Preparation

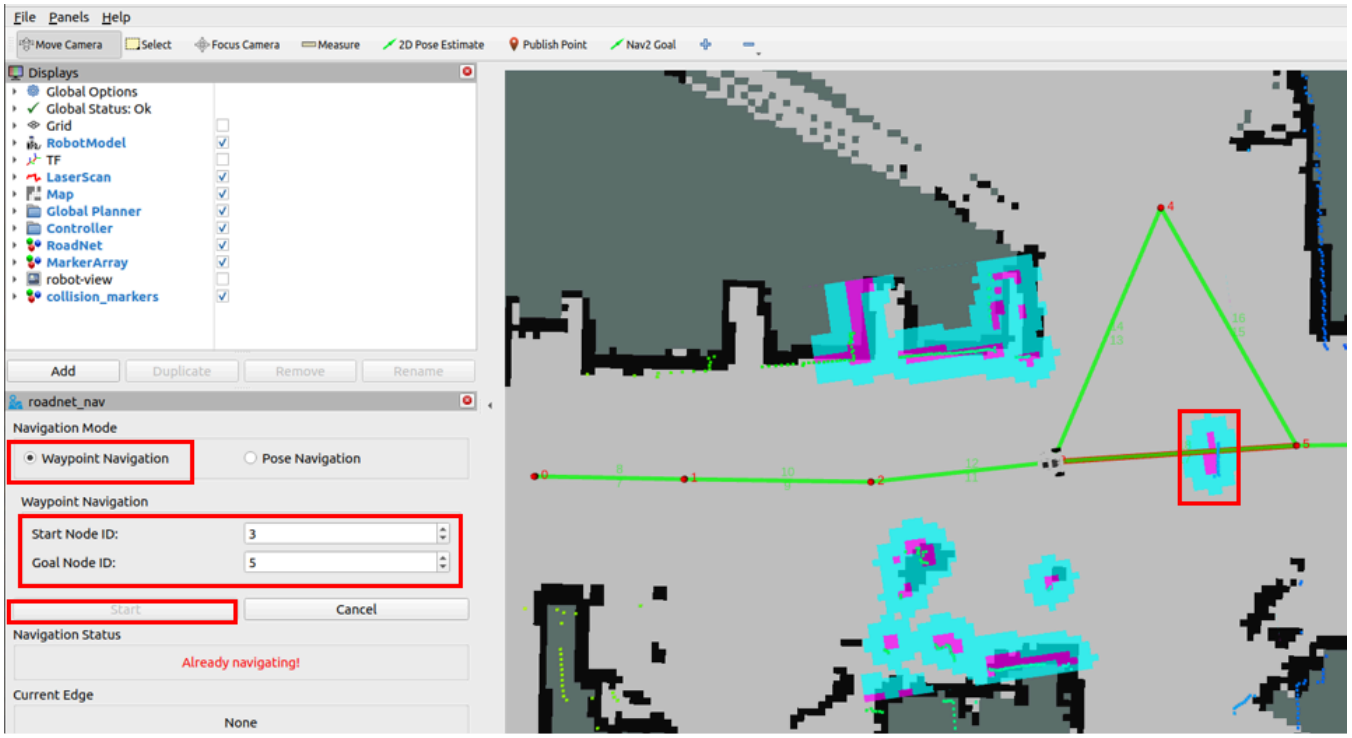
```
ros2 launch yahboomcar_bringup roadnet_nav.launch.py
```

- Click **2D Pose Estimate** to initialize the robot's pose on the map



3. Path Replanning Case Demonstration

- Here we take waypoint-to-waypoint navigation as an example: navigate from waypoint 3 to waypoint 5 through the road network, and place an obstacle in advance on the road in between.



- When an obstacle is detected within 0.5 meters in front of **base_link**, path replanning is triggered, and another route 3→4→5 is chosen to reach the goal waypoint.
- The obstacle is displayed and marked on the map as a red sphere.



4. Source Code Analysis

- Source code path of the implementation program:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/roadnet/src/route_server.cpp
```

- Obtain the Action goal request `goal`, and initialize the result, feedback, and replanning state objects
- **Outer loop** (replanning loop):
    - Validate the legitimacy of the request, check whether a new goal has preempted it, and clear residual costs in the global costmap
    - Call `findRoute` to compute a feasible path in the road network, and call `densify` to densify the path
    - Wait for the `FollowPath` Action server to be ready, send the dense path, and begin tracking
- **Inner loop** (20 Hz path tracking):
    - Check for cancel/preempt requests, and check whether `FollowPath` has completed
    - Obtain the robot's current pose via TF, and compute the distance to the next waypoint
    - When the distance < `radius_threshold`, determine that the waypoint has been reached and switch to the next edge
    - Call `OperationsManager` to perform obstacle detection and replanning judgment
    - When a collision is detected, cancel the current `FollowPath`, set `need_reroute` to break out of the inner loop, and the outer loop replans the path

```
  void RouteServer::computeRouteAndFollow()
  {
    RCLCPP_INFO(get_logger(), "Computing route and following path to goal.");

    auto goal = compute_route_and_follow_server_->get_current_goal();
    auto result = std::make_shared<RoadNetToGoalResult>();
    auto feedback = std::make_shared<RoadNetToGoalFeedback>();
    ReroutingState rerouting_info;
    auto start_time = this->now();

    try
    {
      Route route;
      nav_msgs::msg::Path path;
// Reroute trigger conditions: (obstacle detection) or a request to the rerouting service
(ReroutingService)
      while (rclcpp::ok())
      {
        if (!isRequestValid(compute_route_and_follow_server_))
        {
          return;
        }
        // Check whether a new goal has preempted the current one
        if (compute_route_and_follow_server_->is_preempt_requested())
        {
```

```
          RCLCPP_INFO(get_logger(), "A new request is received, and the target of the
original request is preempted");
          goal = compute_route_and_follow_server_->accept_pending_goal();
          rerouting_info.reset();
        }
        // Clear residual costs in global_costmap before planning to avoid affecting
planning
        clearGlobalCostmap();

        // Compute a feasible route in the road network
        route = findRoute(goal, rerouting_info);
        RCLCPP_INFO(get_logger(), "Route found with %zu nodes and %zu edges",
                    route.edges.size() + 1u, route.edges.size());
        path = path_converter_->densify(route, rerouting_info, route_frame_, this->now());

        // Print the path
        std::string route_str;
        if (route.start_node)
        {
          route_str = std::to_string(route.start_node->nodeid);
        }
        for (const auto &edge : route.edges)
        {
          route_str += " ---------->> " + std::to_string(edge->end->nodeid);
        }
        RCLCPP_INFO(get_logger(), "Find route plan: %s", route_str.c_str());

        // Wait for FollowPath action server
        if (!follow_path_client_->wait_for_action_server(std::chrono::seconds(5)))
        {
          RCLCPP_ERROR(get_logger(), "FollowPath action server not available!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath action server not available";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }
        // Send the path to FollowPath
        auto follow_goal = FollowPath::Goal();
        follow_goal.path = path;

        auto send_goal_options = rclcpp_action::Client<FollowPath>::SendGoalOptions();
        auto goal_handle_future = follow_path_client_->async_send_goal(follow_goal,
send_goal_options);

        // Wait for goal acceptance
        if (goal_handle_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal send timed out!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath goal send timed out";
          compute_route_and_follow_server_->terminate_current(result);
          return;
```

```
        }

        auto follow_goal_handle = goal_handle_future.get();
        if (!follow_goal_handle)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal was rejected!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath goal rejected by controller";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }
        RCLCPP_DEBUG(get_logger(), "FollowPath goal accepted, robot is moving!");

        // Get result future for polling
        auto follow_result_future = follow_path_client_-
>async_get_result(follow_goal_handle);

        // Track path progress and detect obstacles
        double radius_threshold = 2.0;
        nav2_util::declare_parameter_if_not_declared(
            shared_from_this(), "radius_to_achieve_node",
            rclcpp::ParameterValue(radius_threshold));
        radius_threshold = get_parameter("radius_to_achieve_node").as_double();

        EdgePtr current_edge = nullptr;
        NodePtr next_node = nullptr;
        int edge_idx = -1;
        RouteTrackingState tracking_state;
        tracking_state.last_node = route.start_node;
        tracking_state.next_node = route.start_node;

        // Advance to first edge
        if (!route.edges.empty())
        {
          edge_idx = 0;
          current_edge = route.edges[0];
          next_node = current_edge->end;
          tracking_state.current_edge = current_edge;
          RCLCPP_INFO(get_logger(),
                      "Entering edge:%u  id:%u ---------------------> id:%u",
                      current_edge->edgeid,
                      current_edge->start->nodeid,
                      current_edge->end->nodeid);
        }
        rclcpp::Rate rate(20.0);
        bool follow_completed = false;
        bool route_tracking_done = route.edges.empty();
        bool need_reroute = false;

        while (rclcpp::ok() && !follow_completed && !need_reroute)
        {
          // Check if our action was cancelled
          if (compute_route_and_follow_server_->is_cancel_requested())
```

```
          {
            RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            compute_route_and_follow_server_->terminate_all();
            return;
          }
          // Check if preempted by a new goal
          if (compute_route_and_follow_server_->is_preempt_requested())
          {
            RCLCPP_INFO(get_logger(), "Goal preempted, cancelling FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            break;
          }
.........
```

5. Parameter Debugging

- Road network navigation parameter file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/roadnet_nav.yaml
```

- **Enable/disable the replanning feature**: modify the `operations` list
  - `operations: ["ReroutingService", "CollisionMonitor"]`: enable both the external rerouting service and collision-detection replanning (default)
  - `operations: ["ReroutingService"]`: enable only the external rerouting service, disable collision detection; during navigation there is no automatic obstacle avoidance or replanning
  - `operations: []`: disable everything; no replanning is triggered
- **Adjust the collision detection distance**: modify `max_collision_dist` under `CollisionMonitor`. The unit is meters, representing the maximum distance in front of base_link for detecting obstacles; the smaller the value, the closer the detection range
- **Adjust the node achievement radius**:
  - `radius_to_achieve_node`: the achievement judgment radius for intermediate path nodes; when the robot's distance to the waypoint is smaller than this value, it is judged to be reached
  - `boundary_radius_to_achieve_node`: the achievement judgment radius for the start/goal points
- **Other CollisionMonitor parameters**:
  - `rate`: collision detection frequency (Hz); the higher the value, the more timely the detection but the greater the CPU usage
  - `reroute_on_collision`: whether to trigger replanning when a collision is detected
  - `max_cost`: maximum allowed cost value; areas exceeding this value are treated as obstacles
  - `check_resolution`: check resolution; 1 means checking cell by cell

```
route_server:
  ros__parameters:
```

```yaml
    # Path generation
    route_frame: "map"
    path_density: 0.05

    tracker_update_rate: 3.0
    aggregate_blocked_ids: false
    boundary_radius_to_achieve_node: 0.3          # Achievement radius threshold for the path (start / goal)
    radius_to_achieve_node: 0.35                   # Achievement radius threshold for intermediate path nodes (non-start / non-goal)
    # Nearest neighbor search
    enable_nn_search: false              # Whether to enable a breadth-first search based on the costmap to find the nearest node (when off, only the waypoint with the smallest Euclidean distance is used; when on, collisions along the path are taken into account)
    max_nn_search_iterations: 10000
    num_nearest_nodes: 5                # Number of nearest neighbor nodes found using the KD tree

    smooth_corners: false                         # Whether to enable corner smoothing
    #smoothing_radius: 0.2
    #smoothing_angle_threshold: 2.96 # Angle smoothing threshold

    # Path pruning
    max_prune_dist_from_edge: 8.0
    min_prune_dist_from_goal: 0.15
    min_prune_dist_from_start: 0.10
    prune_goal: true

    # Rerouting service
    operations: ["ReroutingService", "CollisionMonitor"]
    # operations: ["ReroutingService"]
    ReroutingService:
      plugin: "roadnet::ReroutingService"
    CollisionMonitor:
      plugin: "roadnet::CollisionMonitor"
      costmap_topic: "global_costmap/costmap_raw"  # Do not modify
      rate: 1.0                    # Check frequency (Hz)
      reroute_on_collision: true   # Trigger replanning when a collision is detected
      max_cost: 253.0              # Maximum allowed cost
      check_resolution: 2          # Check resolution (1 = every cell)
      max_collision_dist: 0.5      # Maximum check distance (meters)

    # Edge cost functions
    # edge_cost_functions: ["CostmapScorer","DynamicEdgesScorer"]
    edge_cost_functions: ["DynamicEdgesScorer"]
    CostmapScorer:
      plugin: "roadnet::CostmapScorer" # Costmap scorer
      use_maximum: false           # Use average cost instead of maximum cost
      invalid_on_collision: true  #  Allow colliding edges, but compute a high cost
      invalid_off_map: false       # Do not mark as invalid when outside the costmap
      max_cost: 253.0              # Maximum valid cost
      check_resolution: 4          # Check resolution
      costmap_topic: "global_costmap/costmap_raw"  # Costmap topic, do not modify
```

```
    weight: 1.0                  # Weight
  DynamicEdgesScorer:
    plugin: "roadnet::DynamicEdgesScorer"
```